

C++ Code Conventions

1. Main Principle

Code should be written to minimize the time required for another developer to understand it.

Prefer clear names, simple structure, and readable formatting over clever or overly compact code.

2. File Layout

2.1 Header Files

Header files should follow a predictable structure.

Recommended order:

```
#pragma once

// System includes
#include <string>
#include <vector>

// Third-party includes
#include <sqlite3.h>

// Project includes
#include "storage/Database.h"

namespace Storage {

// Class declarations

}
```

Rules:

- Every header file must use `#pragma once`.
- Include files in this order:
 - System headers
 - Third-party library headers
 - Project headers
- Prefer including required headers instead of relying on forward declarations.

- Namespaces should wrap the whole header after includes.
 - Inline functions should be used only for small functions, around 10 lines or less.
-

2.2 Source Files

Source files should follow this include order:

```
#include "storage/Database.h"

// System includes
#include <stdexcept>
#include <string>

// Third-party includes
#include <sqlite3.h>

// Project includes
#include "storage/Statement.h"

namespace Storage {

// Function definitions

}
```

Rules:

- Include the corresponding header first.
 - Then include system headers.
 - Then third-party headers.
 - Then project headers.
 - Namespaces should wrap the source file after includes.
 - Avoid unnecessary inline function definitions in `.cpp` files.
-

3. Naming Conventions

3.1 File Names

File names use PascalCase.

```
UserCredentials.h
UserCredentials.cpp
DatabaseConnection.h
DatabaseConnection.cpp
```

Rules:

- File names start with a capital letter.
 - Each new word starts with a capital letter.
 - Do not use underscores.
 - Header files use `.h`.
 - Source files use `.cpp`.
 - In general, every `.cpp` file should have a matching `.h` file.
-

3.2 Type Names

Classes, structs, typedefs, and enums use PascalCase.

```
class UserRepository;  
struct MailMessageRecord;  
enum DeliveryStatus;
```

Rules:

- Start with a capital letter.
- Each new word starts with a capital letter.
- Do not use underscores.
- Do not use prefixes such as `C`, `T`, or `Type`.
- Do not use suffixes such as `_t`.
- Interface classes should be prefixed with `I`.

Example:

```
class IUserRepository  
{  
public:  
    virtual ~IUserRepository() = default;  
};
```

3.3 Variable Names

Variables use lowercase words separated by underscores.

```
std::string user_email;  
int retry_count;  
bool is_active;
```

Rules:

- Use `lower_snake_case`.
- Do not use Hungarian notation.
- Do not use prefixes like `btnCancel`.
- Prefer meaningful names over generic names like `tmp` or `retval`.
- For booleans, use words such as:
 - `is`
 - `has`
 - `can`
 - `should`

Good:

```
bool is_authenticated;  
bool has_attachment;  
bool can_retry;  
bool should_send_bounce;
```

Bad:

```
bool auth;  
bool flag;  
bool not_failed;
```

3.4 Class Data Members

Non-static class data members use `m_` prefix.

```
class Database  
{  
public:  
    explicit Database(const std::string& path);  
  
private:  
    sqlite3* m_connection;  
    std::string m_path;  
};
```

Static class data members use `s_` prefix.

```
class ServerConfig  
{  
private:
```

```
static int s_default_port;
};
```

3.5 Struct Data Members

Struct data members are named like ordinary variables.

```
struct UserRecord
{
    std::int64_t id;
    std::string username;
    std::string email;
    std::string password_hash;
};
```

Do not use `m_` for struct fields.

3.6 Global Variables

Global variables should be rare.

If a global variable is necessary, prefix it with `g_`.

```
Logger g_logger;
```

3.7 Constants

Constants use uppercase letters with underscores.

```
const int MAX_RETRY_COUNT = 5;
const int DEFAULT_SMTP_PORT = 587;
```

3.8 Function Names

Regular functions use PascalCase.

```
void StartServer();  
void StopServer();  
bool AuthenticateUser();
```

Rules:

- Start with a capital letter.
- Each new word starts with a capital letter.
- Do not use underscores.
- Prefer positive names.

Good:

```
bool IsDone();  
bool IsValid();
```

Bad:

```
bool IsNotDone();  
bool IsInvalid();
```

3.9 Accessors and Mutators

Getter and setter functions should match the variable name and use `get_` and `set_`.

```
class User  
{  
public:  
    std::string get_email() const;  
    void set_email(const std::string& email);  
  
private:  
    std::string m_email;  
};
```

3.10 Namespace Names

Namespaces use PascalCase.

```
namespace Storage {  
namespace Sntp {  
namespace Account {
```

3.11 Enumerator Names

Enumerators may use either constant-style naming or PascalCase.

Acceptable:

```
enum class DeliveryStatus  
{  
    Queued,  
    Processing,  
    Delivered,  
    Bounced,  
    Failed  
};
```

Also acceptable:

```
enum DeliveryStatus  
{  
    DELIVERY_STATUS_QUEUED,  
    DELIVERY_STATUS_PROCESSING,  
    DELIVERY_STATUS_DELIVERED  
};
```

Prefer `enum class` for modern C++ code.

3.12 Macro Names

Avoid macros when possible.

If macros are required, use uppercase letters with underscores.

```
#define STORAGE_VERSION_MAJOR 1
```

4. Formatting

4.1 Line Length

A line should fit comfortably on the screen.

Avoid extremely long lines, especially when they reduce readability.

4.2 Spaces and Tabs

Use tabs for indentation.

Be consistent across the project.

4.3 Function Declarations and Definitions

The return type should usually be on the same line as the function name.

```
bool AuthenticateUser(const std::string& email, const std::string& password)
{
    return true;
}
```

Rules:

- Opening curly brace goes on a new line.
- No space between the function name and opening parenthesis.
- No spaces inside the parameter parentheses.
- Parameter order should be:
 - Input parameters
 - Output parameters

Example with many parameters:

```
std::int64_t CreateMessage(
    const std::string& sender_email,
    const std::string& subject,
    const std::string& body,
    std::string& error_message
)
{
    // ...
}
```

4.4 Conditionals and Loops

Use a space between control keywords and opening parenthesis.

```
if (is_authenticated)
{
    SendMessage();
}
```

Rules:

- Use no spaces inside parentheses.
- Opening curly brace goes on a new line.
- Short single-line statements may omit braces if readability is improved.
- If one branch of an `if-else` uses braces, all branches should use braces.

Good:

```
if (is_valid)
{
    SaveMessage();
}
else
{
    RejectMessage();
}
```

Avoid mixing styles:

```
if (is_valid)
{
    SaveMessage();
}
else
    RejectMessage();
```

4.5 Lambda Expressions

Use the same formatting rules as functions and conditionals.

```
auto handler = [](const MailMessage& message)
{
```

```
    return message.IsValid();  
};
```

4.6 Pointer and Reference Expressions

Prefer placing `*` and `&` adjacent to the type.

```
Database* database;  
const std::string& email;
```

Rules:

- No spaces around `.` or `->`.
- No space after `*` or `&`.

Good:

```
user->SendMessage();  
message.subject;
```

4.7 Boolean Expressions

If a boolean expression is long, break it consistently.

```
if (  
    is_authenticated &&  
    has_valid_sender &&  
    can_send_message  
)  
{  
    AcceptMessage();  
}
```

4.8 Return Values

Do not wrap return expressions in unnecessary parentheses.

Good:

```
return is_valid;
```

Bad:

```
return (is_valid);
```

4.9 Namespace Formatting

Do not indent namespace contents.

```
namespace Storage {  
  
class Database  
{  
public:  
    void Open();  
};  
  
}
```

4.10 Preprocessor Directives

The `#` symbol should start at the beginning of the line.

```
#ifdef ENABLE_LOGGING  
#include "Logger.h"  
#endif
```

5. Class Format

Classes should be organized consistently.

Recommended order:

```
class UserRepository  
{  
public:  
    // Typedefs and enums
```

```

    // Constants

    // Constructors

    // Destructor

    // Operators

    // Methods

protected:
    // Protected methods/data

private:
    // Private methods

    // Data members
};

```

Rules:

- Opening curly brace goes on a new line.
- Base class names should be on the same line as the class name when possible.
- Access keywords `public:`, `protected:`, and `private:` should not be indented.
- Do not add a blank line immediately after access keywords.
- Prefer this access order:
 - `public`
 - `protected`
 - `private`
- Multiple `public`, `protected`, or `private` sections are allowed if it improves readability.

Example:

```

class UserRepository : public IUserRepository
{
public:
    explicit UserRepository(Database& database);
    ~UserRepository() override = default;

    std::optional<UserRecord> FindByEmail(const std::string& email);

private:
    Database& m_database;
};

```

For multiple base classes:

```
class MailService :
    public IMessageService,
    public IQueueService
{
public:
    void ProcessMessage();
};
```

6. Constructor Initializer Lists

Short initializer lists may be placed on one line.

```
Database::Database(const std::string& path) : m_path(path)
{
}
```

Longer initializer lists should be split across lines.

```
UserRepository::UserRepository(
    Database& database,
    Logger& logger
) :
    m_database(database),
    m_logger(logger)
{
}
```

7. Comments

7.1 General Rule

Do not comment facts that can be quickly understood from the code.

Good code should explain itself through clear names and structure.

Bad:

```
// Increment retry count
retry_count++;
```

Better:

```
retry_count++;
```

Useful comment:

```
// Remote SMTP servers may temporarily reject messages due to greylisting.  
ScheduleRetry(message_id);
```

7.2 File Comments

File comments are not mandatory.

7.3 Class Comments

External interfaces should have comments explaining their purpose.

```
// Provides user lookup and creation operations backed by SQLite.  
class UserRepository  
{  
    // ...  
};
```

Comments are not mandatory for small internal classes used only inside one module.

7.4 Function Comments

Functions in external interfaces should have comments explaining what they do and how to use them.

```
// Creates a new user account and returns the generated user id.  
// Throws std::runtime_error if the insert fails.  
std::int64_t CreateUser(  
    const std::string& username,  
    const std::string& email,  
    const std::string& password_hash  
);
```

Function comments are not mandatory for private/internal helper methods when the function name is self-explanatory.

7.5 Implementation Comments

Do not repeat in `.cpp` comments what is already documented in the header.

Use inline comments only when they clarify intent, non-obvious decisions, or named arguments.

```
CreateRetryJob(  
    message_id,  
    recipient_id,  
    /*delay_seconds=*/300  
);
```

8. Project-Specific Recommendations

For this mail server/client project, use the following conventions.

8.1 Directory Structure

```
libs/  
  storage/  
    include/storage/  
    src/  
    migrations/  
  
apps/  
  smtp_server/  
    src/  
  
tests/  
  storage/  
  smtp/
```

8.2 Storage Layer Naming

Recommended files:

```
Database.h  
Database.cpp  
Statement.h  
Statement.cpp  
Transaction.h  
Transaction.cpp  
MigrationRunner.h  
MigrationRunner.cpp
```

```
UserRepository.h
userRepository.cpp
MailRepository.h
MailRepository.cpp
QueueRepository.h
QueueRepository.cpp
```

8.3 SQL Migration Naming

Use numbered migration files:

```
001_initial_schema.sql
002_mail_queue.sql
003_mailboxes.sql
```

Rules:

- Never edit old migrations after they are applied.
- Add a new migration for every schema change.
- Migration files should be stored in:

```
libs/storage/migrations/
```

8.4 Database Code Style

Keep raw SQLite calls inside `libs/storage`.

Good:

```
UserRepository users(database);
auto user = users.FindByEmail(email);
```

Bad:

```
sqlite3_exec(...); // directly inside SMTP session code
```

SMTP, IMAP, and account services should use repositories instead of raw SQL.

9. Summary

Core rules:

- Prefer readable code over clever code.

- Use PascalCase for files, types, functions, and namespaces.
- Use `lower_snake_case` for variables.
- Use `m_` for class data members.
- Use `s_` for static class data members.
- Use uppercase with underscores for constants and macros.
- Use `#pragma once` in headers.
- Keep includes ordered and explicit.
- Use tabs for indentation.
- Keep comments useful, not redundant.
- Keep database access inside repository/storage classes.